

Phase 0 Review Packet

The production foundation: monorepo, database, authentication, scoped permissions, versioned consent, audit log, double-entry ledger and the payment abstraction. What was executed, what was not, and what is deliberately absent.

Branch **claude/create-app-repository-1cxsih**

Commit **87743b6**

139 files · +14 468 / -7 049

Tests **102 passed, 3 skipped**

Lint **clean**

Build **green**

iPayMoney **not integrated**

HOW TO READ THIS PACKET

Three states are kept strictly apart throughout. **Tested** means a test was executed and its result is reproduced below. **Not tested** means the code exists but no automated test covers it. **Not implemented** means the code does not exist. Nothing is called "verified" unless the relevant test actually ran.

PHASE 0 ITEM	STATE	EVIDENCE
1 · Monorepo foundation	TESTED	pnpm workspace, 5 packages + 1 app; lint, typecheck, test, build all run across it
2 · Application structure	TESTED	eslint boundary rule forbids <code>packages/*</code> importing Next.js; build green
3 · PostgreSQL	TESTED	PostgreSQL 16.13; integration tests run against a real database
4 · Database migrations	TESTED	2 migrations applied; 22 tables; schema rebuilt from scratch in CI
5 · ORM setup	TESTED	Drizzle; every integration test goes through it
6 · Authentication foundation	PARTLY TESTED	Domain core tested (23 tests). No HTTP routes, no sign-in screen.
7 · Phone OTP architecture	PARTLY TESTED	Generation, hashing, expiry, attempts, binding tested. No SMS provider.
8 · Email authentication architecture	PARTLY TESTED	scrypt hashing and verification tested. No email sending.
9 · Secure sessions	PARTLY TESTED	Token issuance, hashing, expiry, revocation, elevation tested. No cookie wiring.
10 · Scoped RBAC	TESTED	11 tests including scope confinement and revocation
11 · Permission system	TESTED	22 permissions, 7 roles, seeded and asserted
12 · i18n foundation	NOT TESTED	fr/en catalogues and resolver exist. No route-level wiring, no tests.
13 · Design system foundation	NOT TESTED	Tokens extracted to <code>packages/ui</code> ; app consumes them. No component tests.
14 · CI	NOT TESTED	Workflow written and committed. Never executed on GitHub — see limitations.
15 · Lint	TESTED	Exit 0 across all 6 projects
16 · Type checking	TESTED	Exit 0 across all 6 projects, strict mode
17 · Automated test framework	TESTED	Vitest + fast-check; 105 specs
18 · Database test environment	TESTED	Separate <code>TEST_DATABASE_URL</code> ; suite refuses to run without it
19 · Environment configuration	NOT TESTED	<code>.env.example</code> committed, <code>.env</code> ignored
20 · Secrets handling	TESTED	Redaction by key name tested; no secrets in the repository
21 · Observability / logging	TESTED	5 tests incl. redaction, levels, child context

PHASE 0 ITEM	STATE	EVIDENCE
22 · Audit-log foundation	PARTLY TESTED	Table and writer exist; append-only enforcement tested. Writer itself has no direct test.
23 · Initial financial schema	TESTED	14 ledger tests incl. property-based sequences
24 · Payment-provider abstraction	TESTED	13 tests; mock refuses to load in production
25 · iPayMoney adapter architecture	NOT IMPLEMENTED	Boundary throws; 3 sandbox specs skipped. Blocked on documentation.

2

Verify it yourself

Needs Node 22+, pnpm 10+, and a PostgreSQL 16 you can create two databases in. No credentials, no third-party accounts, no network beyond the npm registry.

1

Check out the commit

```
git clone https://github.com/abdoulasani/Afrinext.git
cd Afrinext
git checkout 87743b6
pnpm install
```

2

Point it at a database

`TEST_DATABASE_URL` must differ from `DATABASE_URL`. The suite truncates every table it touches and refuses to start otherwise.

```
createdb afrinext && createdb afrinext_test

cp .env.example .env
# then set, for a local server on the default port:
#   DATABASE_URL=postgres://postgres@127.0.0.1:5432/afrinext
#   TEST_DATABASE_URL=postgres://postgres@127.0.0.1:5432/afrinext_test
```

3

Migrate and seed

```
pnpm db:migrate
pnpm --filter @afrinext/core seed
```

Expected: Migrations complete. then Seeded 7 currencies, 11 countries, 22 permissions, 7 roles, 9 settings, 7 legal documents (placeholder texts).

4

Run the whole verification chain

```
pnpm lint
pnpm typecheck
pnpm test
pnpm build
```

All four must exit 0. Actual output is reproduced in section 3.

5 Prove the ledger balances over a real cycle

Posts a capture, a provider settlement, a sale settlement with a referral, a payout approval and a payout, then reconciles. This is the gate CI runs.

```
pnpm --filter @afrinext/core exec tsx scripts/reconcile-check.ts
```

6 Check the app still runs, and reaches the database

```
pnpm build && pnpm start &  
curl -s localhost:3000/api/health
```

7 Try to break the ledger by hand

Worth doing yourself rather than trusting the tests. Each of these must be refused by the database, not by application code:

```
-- append-only  
update ledger_entries set amount_minor = 1;      -- ERROR: append-only  
delete from ledger_entries;                      -- ERROR: append-only  
  
-- unbalanced transaction, rejected at COMMIT  
begin;  
insert into ledger_transactions (id, kind) values (gen_random_uuid(), 'adjustment');  
-- ... insert a single entry against that transaction ...  
commit;                                           -- ERROR: does not balance
```

Results, as executed

Copied from the run on commit 87743b6 against PostgreSQL 16.13. Nothing here is predicted.

Tests — 102 PASSED • 3 SKIPPED

```

✓ src/ledger/ledger.test.ts          (14 tests) 4381ms
✓ src/auth/auth.test.ts              (23 tests) 1978ms
✓ src/authz/authorize.test.ts        (11 tests) 902ms
✓ src/payments/payments.test.ts      (13 tests) 11ms
✓ src/money/allocate.test.ts         (9 tests) 184ms
✓ src/consent/consent.test.ts        (6 tests) 464ms
✓ src/money/money.test.ts            (13 tests) 128ms
✓ src/observability/logger.test.ts   (5 tests) 7ms
✓ src/money/currency.test.ts         (7 tests) 10ms
✓ src/payments/ipaymoney.integration.test.ts
                                     (4 tests | 3 skipped) 3ms

```

Test Files 10 passed (10)

Tests 102 passed | 3 skipped (105)

The 3 skipped specs are the iPayMoney sandbox tests. They skip unless `IPAYMONEY_BASE_URL` and `IPAYMONEY_API_KEY` are set, so CI stays green without credentials and turns red the moment a real sandbox run regresses.

Lint and typecheck — EXIT 0

```

packages/db    lint: Done    packages/db    typecheck: Done
packages/i18n  lint: Done    packages/i18n  typecheck: Done
packages/ui    lint: Done    packages/ui    typecheck: Done
packages/core  lint: Done    packages/core  typecheck: Done
apps/web       lint: Done    apps/web       typecheck: Done

```

TypeScript runs with `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`, `noUnusedLocals` and `verbatimModuleSyntax` — set from the first commit, because retrofitting them into a large codebase is miserable.

Build — GREEN

```

✓ Compiled successfully
  Finished TypeScript in 3.0s

```

Route (app)

```

r f /
  o /_not-found
  f /api/health
  f /api/listings
  f /api/listings/[id]
  f /browse
  f /listing/[id]
  o /offline
  o /saved
  o /submit

```

Ledger reconciliation gate — CLEAN

A full capture → provider settlement → sale settlement → payout approval → payout, then reconciliation:

```
{
  "scenario": {
    "gross": "10000", "seller": "8200",
    "platform": "1620", "referral": "180"
  },
  "checkedAccounts": 8,
  "drift": 0,
  "unbalancedTransactions": 0,
  "netByCurrency": { "XOF": "0" }
}
Ledger reconciliation clean over a real capture → settlement → payout cycle.
```

$8\,200 + 1\,620 + 180 = 10\,000$ exactly. The referral is 10% of Afrinext's 1 800 commission, not of the gross, so it can never exceed what was earned on the sale.

Health endpoint — REACHABLE

```
$ curl -s localhost:3000/api/health
{ "status": "ok", "database": "reachable",
  "migrationsApplied": 2, "currenciesSeeded": 7,
  "permissionsSeeded": 22, "paymentProvider": "mock", "latencyMs": 2 }
```

This is the end-to-end proof that the monorepo wiring works: the Next.js route calls `packages/core`, which calls `packages/db`, which reaches PostgreSQL — and the structured logger emits it.

ON SCREENSHOTS

None are included, deliberately. Phase 0 shipped **no new user-facing screens** — the app served is still the original directory prototype. A screenshot of it would suggest progress that did not happen. The terminal output above is the evidence that matters here.

Architecture changes

The blueprint is now revision 2. Four things changed, all of them reflecting confirmed business decisions rather than engineering opinion.

CHANGE	WHERE
New section 01 — company, contract and regulatory posture. Records AFRI NEXT TECHNOLOGIE as operator, and separates seven layers that get conflated: company registration (A), contract with users (B), payment-provider relationship (C), internal ledger (D), seller entitlement (E), payout operation (F) and regulatory authorization (G). States plainly that NIF and RCCM establish A only, and that nothing in A–F establishes G.	blueprint §01
Consent architecture. Versioned documents, append-only acceptance records, and a server-side gate. Consent establishes B, never G.	blueprint §01
Confirmations. Referral single-level, V1/V2/V3 scope, Niger + XOF as launch market — all moved from "recommendation" to "confirmed". V2 and V3 architecture retained deliberately.	§00, §H, §M
iPayMoney section rewritten. Confirmed as provider with a merchant account; API unverified and unreachable; 13 documentation items listed.	§P

THE CONSEQUENCE IN CODE

Because layer G is unresolved, three things are true of the implementation and must stay true: **the ledger records entitlement, not custody** — nothing asserts where money physically sits; **the payout rail is an interface**, so the answer can change without a rewrite; and **no field or screen calls a seller balance a deposit or protected funds**.

Database schema and migrations

22 tables, 2 migrations, 6 triggers, 1 view. Migration 0000 is generated from the Drizzle schema; 0001 is hand-written and is the one worth reading closely.

GROUP	TABLES
Reference	currencies, countries, locales
Identity	users, user_identities, sessions, otp_challenges, rate_limit_counters
Authorization	permissions, roles, role_permissions, role_assignments
Consent	legal_documents, legal_document_versions, consent_records
Ledger	ledger_accounts, ledger_transactions, ledger_entries, account_balances, idempotency_keys
Operations	audit_logs, platform_settings

What migration 0001 adds, and why it is not in application code

GUARANTEE	MECHANISM	TESTED
Every transaction balances, per currency	<code>CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED</code> — evaluated at COMMIT; so all entries can be inserted first	YES
An entry's currency matches its account	<code>BEFORE INSERT</code> trigger	YES
Ledger entries and transactions are append-only	<code>BEFORE UPDATE OR DELETE</code> trigger that raises	YES
Consent records and audit logs are append-only	Same	YES (CONSENT)
Least-privilege application role	<code>afrinext_app</code> role with UPDATE/DELETE revoked on the immutable tables	NOT TESTED — dev connects as superuser
Authoritative balance for reconciliation	<code>ledger_account_balances_derived</code> view	YES

A trigger is used rather than only a `REVOKE` because a superuser ignores `REVOKE`, and development connections often are superuser. The revoke is defence in depth behind it.

WORTH YOUR ATTENTION

Drizzle has no down-migrations. CI compensates by dropping the schema and rebuilding it from zero on every run, which proves the migrations apply cleanly to an empty database — but it is **not** the same as being able to roll back a release in production. If you want true reversibility, say so and it becomes a Phase 1 task.

READ THIS FIRST

Nobody can sign in yet. Phase 0 built the authentication *domain core* — the primitives, tested in isolation. There are no HTTP routes, no cookie handling, no SMS or email delivery, and no screens. Wiring lands in Phase 1.

PIECE	STATE	DETAIL
Password hashing	TESTED	scrypt ($N=2^{16}$, $r=8$, $p=1$). Encoded hash carries its own parameters, so the algorithm can change without invalidating existing hashes. <code>needsRehash()</code> flags weak ones.
Session tokens	TESTED	32 random bytes; only the SHA-256 is stored, so a database leak does not hand over live sessions. Constant-time comparison.
Session lifecycle	TESTED	Expiry, revocation, and a 10-minute elevation window for payout-sensitive actions.
OTP	TESTED	Cryptographically uniform 6 digits, hashed with identifier and purpose as salt, bounded attempts, 5-minute expiry. A code issued for sign-in will not verify for step-up, or for a different phone number.
Phone normalisation	TESTED	E.164 only, calling codes from the <code>countries</code> table, longest-prefix match. Property test: any input either yields valid E.164 or an explicit failure.
OTP rate limiting	NOT IMPLEMENTED	The <code>rate_limit_counters</code> table exists; nothing writes to it yet. Bounding attempts alone does not stop an attacker who can request unlimited fresh codes — this is a Phase 1 must.
SMS / email delivery	NOT IMPLEMENTED	No provider chosen. The OTP core is provider-agnostic.
HTTP routes, cookies, screens	NOT IMPLEMENTED	Phase 1.

Authorization status

Scoped role assignments, one gate, 22 permissions across 7 roles. **11 TESTS**

There is **no role column on users**. Roles are rows in `role_assignments` carrying a scope type and id, so one person being a buyer, an instructor, a rider and staff on someone else's store at the same time costs nothing.

BEHAVIOUR	TESTED
A user with no roles is denied everything	YES
A scoped role does not leak into another scope — store owner of A cannot publish to store B	YES
A global grant satisfies a scoped check	YES
A revoked assignment stops being honoured	YES
Permissions accumulate across several roles	YES
Nobody can approve their own withdrawal, whatever roles they hold	YES
<code>superadmin</code> deliberately lacks <code>withdrawal.approve</code>	YES
<code>support</code> and <code>ops</code> hold no financial capability	YES

Administering roles and moving money are separate capabilities by design: `superadmin` can grant roles but cannot approve a payout, and `finance` can approve payouts but cannot change who holds which role. That split is only meaningful if nobody is granted both — an operational rule the console will need to surface.

The part worth most of your review time. **14 LEDGER TESTS · 29 MONEY TESTS**

Money

Money is `{ amountMinor: bigint, currency: string }`. No floats anywhere. The minor-unit exponent is read from the `currencies` table, never assumed — **XOF and XAF have zero decimal places**, so code that divides by 100 is wrong by a factor of 100 across the whole UEMOA zone. A test asserts this directly.

The split rule

Every non-residual line is rounded half-up; the seller's share is `gross -  $\Sigma(\text{other lines})$` , never computed independently. So a split always re-sums to the gross and the remainder consistently favours the seller.

Referral commission is drawn **from Afrinext's own commission**, not from the gross. Property tests over 2 000 randomised cases assert:

- the split re-sums to the gross exactly, whatever the rates;
- no share is ever negative;
- **the referral never exceeds what the platform earned** — the confirmed guardrail;
- the rounding remainder lands with the seller, not the platform.

Invariants proved by test

INVARIANT	HOW IT IS PROVED
Every transaction balances per currency	Deferred DB trigger; a hand-crafted unbalanced insert is rejected at COMMIT
Entries are immutable	UPDATE and DELETE both raise; asserted directly
The whole ledger nets to zero per currency	Asserted after every operation in the property test
Cached balance equals the sum of entries	Asserted after every operation; drift injected by hand is detected
Duplicate operations are harmless	Same idempotency key replays and returns the original transaction
A key reused with different inputs is refused	Throws <code>IdempotencyConflictError</code> rather than replaying someone else's result
Reversals restore state without editing history	Balances return to zero; the original entries still exist alongside the mirror
Money never disappears on a failed payout	Funds return to the user's available balance

The property test

25 randomised runs of 4–14 operations each — captures, settlements with and without referral, hold releases, payout approvals and settlements, and reversals — across three sellers and an ambassador. **After every single operation** it asserts: no unbalanced transaction exists, the ledger nets to zero per currency, and every cached balance equals its derived balance.

The test distinguishes a deliberate domain refusal from a broken invariant: a refused operation is allowed, anything else re-throws and fails the property.

WHAT THE LEDGER TESTS DO NOT COVER

- **Concurrency.** Every test is single-threaded. The balance cache uses an atomic increment under the row lock taken by `ON CONFLICT DO UPDATE`, which should serialise concurrent posts — but that reasoning is **not tested**. A concurrent-posting test belongs in Phase 1.
- **Multi-currency transactions.** Every test uses XOF. The schema and code handle several currencies per transaction; nothing exercises it.
- **Scale.** Largest run is a few hundred entries. No behaviour is known at millions of rows.

IPAYMONEY IS NOT INTEGRATED

Confirmed as the provider, and Afrinext holds a merchant account — but the official documentation is not in the repository and the provider's domain is **not reachable from the build environment** (egress blocked by policy). So nothing about the API has been verified: not the authentication scheme, not an endpoint, not a payload, not a webhook signature.

`IPayMoneyProvider` exists as a boundary and **every method throws `ProviderNotConfiguredError`**, naming what is missing. A stub returning plausible data would let the system appear to work and fail in production, on money.

PIECE	STATE
<code>PaymentProvider</code> interface	TESTED Provider-agnostic. <code>createPayout</code> is optional because whether iPayMoney offers disbursement is unknown; <code>supportsPayouts()</code> is how callers check.
<code>MockPaymentProvider</code>	TESTED Charges, idempotency, refunds, payouts, and HMAC webhook verification against the raw body with constant-time comparison. Tampered bodies and bad signatures both rejected.
Production safety	TESTED The mock throws if constructed when <code>NODE_ENV=production</code> , and the registry refuses to hand it out. A misconfigured environment stops the process rather than quietly accepting payments that never happened.
Unknown provider id	TESTED Refused rather than silently falling back
iPayMoney adapter	NOT IMPLEMENTED Boundary only
Sandbox integration specs	SKIPPED 3 specs, gated on credentials in the environment

The single most useful thing you can send is iPayMoney's official documentation — the 13 items are listed in `docs/providers/ipaymoney/README.md`. Items 6 (webhook signature), 7 (whether XOF is sent as whole francs) and 8 (whether a payout API exists) most often invalidate an assumed design.

CONTROL	STATE	NOTE
Passwords never stored in the clear	TESTED	script with per-hash salt; parameters encoded in the hash
Session tokens never stored in the clear	TESTED	SHA-256 only; constant-time comparison
OTP codes never stored in the clear	TESTED	Salted by identifier and purpose, so one rainbow table of six-digit strings does not work
Secrets never reach the logs	TESTED	Redaction by key name, case- and punctuation-insensitive, recursive
No secrets in the repository	TESTED	<code>.env</code> git-ignored; <code>.env.example</code> holds no values
Separation of duty on payouts	TESTED	Self-approval blocked in code
IDOR prevention	PARTLY	Authorization scopes queries in SQL rather than checking after fetch. Untested at the HTTP layer, because there are no protected routes yet.
Webhook signature verification	MOCK ONLY	The pattern is correct — verify the raw body before parsing, constant time. The real scheme is unknown.
Idempotency on money operations	TESTED	Request hash stored; key reuse with different inputs refused
Audit log	PARTLY	Table and writer exist and the table is append-only. No call sites yet — nothing is being audited in practice.
Rate limiting	NOT IMPLEMENTED	Table exists, nothing writes to it
CSRF, security headers, CSP	NOT IMPLEMENTED	No authenticated routes yet to protect
Dependency vulnerability scanning	NOT IMPLEMENTED	Not in CI. Worth adding in Phase 1.
Penetration test	NOT DONE	Schedule before public launch, not after

STILL TRUE FROM PHASE 1, AND UNCHANGED

The prototype's `POST /api/listings` remains open to anyone — it predates authentication and is scheduled for replacement. It writes only to an in-memory store, so nothing persists, but it should not survive into anything public.

What is NOT implemented

Complete list. Anything not here either works or is named as untested elsewhere in this packet.

AREA	NOT IMPLEMENTED
Authentication	Sign-in, sign-up and sign-out routes; cookie handling; SMS and email delivery; OTP issuance rate limiting; any screen. Nobody can sign in.
Payments	The iPayMoney adapter. Blocked on documentation.
Commerce	Stores, products, courses, cart, checkout, orders, fee-line snapshots. No commerce tables exist yet — deliberately, they belong to Phase 1.
Wallet and payouts	Withdrawal requests, KYC, payout batches, the finance console. The ledger movements exist; the workflow around them does not.
Referral	Attribution, qualification, commission records. The split arithmetic exists and is tested; the programme is not built.
Learn	Courses, lessons, entitlement, video, progress.
Delivery	Everything. V2.
Admin console	No screens. Each phase ships its own admin surface.
i18n	Route-level integration (<code>/fr/...</code> , <code>/en/...</code>) and the provider wiring. Catalogues and the resolver exist; nothing consumes them yet.
Legal document texts	Placeholders only, with content hashes recording them as such. The machinery works with any text; drafting is a legal deliverable.
Audit call sites	The writer exists but nothing calls it. Nothing is being audited yet.
Turborepo	Not added. Plain pnpm scripts are sufficient at 6 projects; revisit when build times justify it.
Deployment	No hosting, no environments, no deploy pipeline. Awaiting the hosting decision.

Known limitations of what *is* built

- **CI has never run.** The workflow is committed but has not executed on GitHub — this environment has no runner. Every step in it was run locally and passes, but the YAML itself is unproven. Expect to fix something on first push.
- **No concurrency testing** anywhere, including the ledger's balance cache.
- **No down-migrations.** CI rebuilds from scratch instead; that is not the same as rolling back a release.
- **The least-privilege role is untested** — local development connects as superuser, so the `REVOKE` s are not exercised. The append-only triggers are, and they bind superusers too.

- **Single currency in practice.** All tests use XOF.
- **The web app is still the prototype.** Its in-memory store is untouched by Phase 0 and still resets on restart.
- **Latency assumption unverified.** eu-west-3 over af-south-1 is still routing intuition, not measurement.

Deviations from the blueprint — your call

Four places where the implementation differs from what the blueprint proposed. Each was a judgement call made to keep Phase 0 moving; each is cheap to reverse now and expensive later.

Deviation 1

Auth core written in-house instead of adopting Better Auth

Why

What Phase 0 needed was session lifecycle, OTP lifecycle and password hashing — roughly 300 lines of framework-free, fully testable code with no dependency to keep patched. Better Auth brings its own session tables that would have to be reconciled with the scoped RBAC and audit design, and its phone-OTP path cannot be exercised without an SMS provider we have not chosen.

What was not written

No cryptography was invented. `script` and SHA-256 come from `node:crypto`; randomness from `randomBytes` and `randomInt`.

Risk

Rolling your own auth is a classic mistake. The mitigation is that this is session *management*, not cryptography, and it is behind an interface — adopting Better Auth later replaces one module.

Your call

Accept, or direct me to adopt Better Auth in Phase 1 before the HTTP layer is built on top.

Deviation 2

script rather than Argon2id

Why

OWASP accepts both. `script` is in `node:crypto` with no native binding to build or audit, and phone OTP — not a password — is the primary credential in this market.

Cost of changing

Low. The encoded hash carries its own parameters, so existing hashes keep verifying while new ones use Argon2id, and `needsRehash()` already flags the old ones.

Note

Parameters are $N=2^{16}$ (~64 MiB per hash) rather than OWASP's suggested 2^{17} . 2^{17} doubles the memory per login and with concurrency becomes a denial-of-service surface. Say the word and it moves.

Drizzle adopted while the ORM question is still open

Why

Phase 0 could not start without an ORM, and the blueprint's stated recommendation was Drizzle. The decision genuinely depends on team composition, which is still unanswered.

Cost of changing now

Moderate: the schema definitions and roughly a dozen query sites. The SQL, the migrations and every invariant in migration 0001 are ORM-independent and would carry over unchanged.

Cost of changing after Phase 1

Considerably higher.

Your call

Tell me the team's Prisma versus raw-SQL experience and I will either confirm or switch — now, not later.

Deviation 4

i18n foundation without route-level wiring

Why

Wiring `/fr/...` and `/en/...` means restructuring every route — and every current route is prototype code scheduled for replacement. Doing it now would be churn against pages that are about to be deleted.

What exists

fr and en catalogues, locale resolution, placeholder interpolation, and the `locales` table. French is the default.

Consequence

Phase 1's first real screens must land with the route wiring, not after it. Flagged so it does not slip.

RISK	SEV	WHERE IT STANDS
Regulatory authorization (layer G). Afrinext receiving customer money and paying sellers may be a regulated activity under BCEAO rules. NIF and RCCM establish company registration only.	HIGH	Unresolved and outside engineering. The architecture reduces exposure the only way it can – the ledger records entitlement rather than asserting custody, and the payout rail is an interface. Gates Phase 6, not Phase 1. Long lead time; worth starting now.
iPayMoney API unknown. Documentation absent and the domain unreachable from the build environment.	HIGH	Contained: the abstraction is built and the mock exercises the whole cycle. Nothing else is blocked. 13 items listed in <code>docs/providers/ipaymoney/README.md</code> .
CI unproven. The workflow has never executed.	MED	Every step passes locally. Expect a fix on first push.
Ledger concurrency untested. The balance cache relies on row-lock serialisation that has been reasoned about, not tested.	MED	Add a concurrent-posting test in Phase 1, before real traffic exists.
Nothing is audited yet. The writer exists but has no call sites.	MED	Must land with the first financial and administrative actions in Phase 1, not after.
OTP issuance is unlimited. Attempts are bounded; requests are not.	MED	Each SMS costs money, so this is a billing risk as well as a security one. Phase 1 must.
No rollback path for migrations.	MED	CI rebuilds from scratch. Decide whether true down-migrations are required.
Legal texts do not exist. Consent machinery works; the documents are placeholders.	MED	Blocks nothing technical. Blocks going live.
No dependency scanning or penetration test.	MED	Add scanning to CI in Phase 1; schedule the test before public launch.
Latency assumption unverified.	LOW	Measure from Niamey before committing to a region.

NOT STARTED

Phase 1 has not begun and will not begin without your explicit approval. This is a proposal.

Phase 1 — identity in use, and the commerce spine · ~3 weeks

Phase 0 built primitives nobody can reach. Phase 1 makes them reachable and puts the first real data behind them.

1. **Authentication over HTTP.** Sign-in, sign-up, sign-out and step-up routes; secure cookie handling; an SMS provider behind an interface; **OTP issuance rate limiting**; the first real screens, with i18n routing wired in from the start.
2. **Audit call sites.** Every authentication event, role change and ledger post writes an audit row in the same transaction.
3. **Consent in the flow.** The gate applied at sign-up and seller onboarding, with real (or at least drafted) document texts.
4. **Stores and the catalogue.** Store creation, digital products and courses as products, categories, media upload.
5. **Commerce tables.** Cart, orders, order items and `order_fee_lines` — the frozen commission snapshot, so a rate change never rewrites history.
6. **Concurrency test** for the ledger, and a dependency scan in CI.

Exit criterion: a real person registers by phone, accepts the terms, creates a store, publishes a paid digital product, and it appears at a public URL — with every step audited and the ledger untouched because nothing has been bought yet.

Payment integration stays gated on the iPayMoney documentation. If it has not arrived by the end of Phase 1, the manual transfer rail from the roadmap's P3 becomes the launch payment method and we should plan for that openly rather than waiting.

Senior developer sign-off checklist

Suggested order. The first four are where a wrong answer is most expensive to discover later.

#	CHECK	WHERE	VERDICT
1	Are the ledger invariants the right set, and is a deferred constraint trigger the right enforcement point? Anything missing?	packages/db/migrations/0001_ledger_invariants.sql	☐ approve ☐ changes ☐ reject
2	Does the property test actually prove what this packet claims? Read it adversarially — could it pass while the ledger is broken?	packages/core/src/ledger/ledger.test.ts	☐ approve ☐ changes ☐ reject
3	Is the residual rule sound, and is drawing referral from platform commission implemented as intended?	packages/core/src/money/allocate.ts	☐ approve ☐ changes ☐ reject
4	Is the posting path correct — idempotency, balance-cache increment, transaction boundaries?	packages/core/src/ledger/post.ts	☐ approve ☐ changes ☐ reject
5	Is <code>authorize()</code> a sufficient gate, and is scope confinement right?	packages/core/src/authz/authorize.ts	☐ approve ☐ changes ☐ reject
6	Is the in-house auth core acceptable, or should Better Auth be adopted before the HTTP layer? (Deviation 1)	packages/core/src/auth/	☐ approve ☐ changes ☐ reject
7	scrypt at $N=2^{16}$, or move to Argon2id? (Deviation 2)	packages/core/src/auth/password.ts	☐ approve ☐ changes ☐ reject
8	Confirm or switch the ORM — decide now, not after Phase 1. (Deviation 3)	packages/db/src/schema/	☐ confirm Drizzle ☐ switch to Prisma
9	Is the schema right? Anything missing that Phase 1 will need, or present that it will not?	packages/db/src/schema/	☐ approve ☐ changes ☐ reject
10	Is the payment abstraction sound, and is throwing the right behaviour for an unverified provider?	packages/core/src/payments/	☐ approve ☐ changes ☐ reject
11	Is the framework boundary drawn in the right place, and worth enforcing by lint?	packages/config/eslint.base.mjs	☐ approve ☐ changes ☐ reject
12	Does the CI workflow cover what it should? It has never run — what will break?	.github/workflows/ci.yml	☐ approve ☐ changes ☐ reject
13	Are down-migrations required before production?	packages/db/migrations/	☐ required ☐ not required

#	CHECK	WHERE	VERDICT
14	Is the Phase 1 proposal the right next step, and its exit criterion the right bar?	section 14	☐ approve ☐ changes ☐ reject

ALSO NEEDED, AND NOT FROM ENGINEERING

- **iPayMoney documentation** — the 13 items. Sandbox credentials are the most useful single thing.
- **Who is advising on layer G** (regulatory authorization), and what is being pursued.
- **Team composition** — settles checklist item 8.
- **Hosting** — managed or containers.
- **SMS provider for Niger** — needed before OTP can actually be delivered in Phase 1.

Phase 0 complete · branch `claude/create-app-repository-1cxsih` · commit `87743b6` · Phase 1 will not begin without explicit approval.